

Query processing

Need a plan to evaluate a query

Plan is a tree of relational algebra operators, with algorithm for each

select * from recipes where date > 1950;

→ use index on date? or scan initial relation?

Search problem:

- what plans?
- how to estimate costs?

(We'll follow System R)

Three main algorithms get used over and over:

- Indexing - using WHERE conditions to get small parts
- Iteration - looping over dataset itself, or just an index
- Partitioning - breaking the data into pieces

Need summary information about relations to make estimates. Keep around, and updated from time-to-time:

- relations: # tuples / # pages
- indexes: # distinct key values / # pages, index height, low/high values

Working out costs

Notation:

t_T = page transfer time = time to transfer block from disk to computer (with no additional time needed to find block)

t_S = page access time = time to seek a page on disk

- sequential access time is for one page is t_T
- random access time for one page is $t_S + t_T$

Ignore CPU time (except when we don't)

Not big-O, we care about actual timing

Specific cost estimates

Linear/Sequential search

- b_r is number of blocks in the relation.
- Assuming file is stored sequentially.

Description	cost
Find starting block	t_S
Read all blocks	$b_r * t_T$
Total	$t_S + b_r * t_T$

- If stored completely randomly (ugh!), $b_r * (t_S + t_T)$.

Which of the following is the best reason as to why cost estimates (like we've been doing) are important?

- A. They allow a database system to provide for the user an estimate of how long a query will take to run.
- B. They allow a database system to provide for the user an estimate of how much memory a query will take to run.
- C. They help a database system administrator figure out if indexes need to be built, or if some other reorganization of the data needs to be made.
- D. They help a database system figure out which approach to use in evaluating a query.

What is the cost for looking up all records that match a search key, when using a primary B+ tree index, when the search key is not unique in the data? Assume the entire relation is stored sequentially on disk. Let

- h_i = height of index
- t_T = block transfer time
- t_S = block seek time
- b_r = number of blocks in file
- b = number of blocks containing search key
- n = number of records containing search key

A. $t_S + b_r * t_T$

B. $(h_i + 1) * (t_S + t_T)$

C. $h_i * (t_S + t_T) + t_S + b * t_T$

D. $(h_i + n) * (t_S + t_T)$

Primary B+-tree index, equality on non-unique key

Description	cost	notes
Navigate to leaf with key	$h_i * (t_S + t_T)$	random
Find first block with key	t_S	
Read all blocks with key	$b * t_T$	clustered, so seq
Total	...	

Suppose we have a secondary B+-tree index where the leaves contain search keys and RIDs, and the RIDs point to the records in a file that is not ordered on the index search key. We'll assume that the number of records that match is relatively small, but not necessarily 1. Which of the following is the best approximation the cost for an equality query using this index?

- h_i = height of index
- t_T = block transfer time
- t_S = block seek time
- b_r = number of blocks in file / relation
- b = number of blocks containing search key
- n = number of records containing search key

A. $t_S + b_r * t_T$

B. $(h_i + 1) * (t_S + t_T)$

C. $h_i * (t_S + t_T) + t_S + b * t_T$

D. $(h_i + n) * (t_S + t_T)$

Secondary B+ tree

Description	cost	notes
Navigate to leaf with key	$h_i * (t_S + t_T)$	random
get block with tuple n times	$n * (t_S + t_T)$	n = tuples matched
Total	$(h_i + n) * (t_S + t_T)$	

“and” query using one index

```
1 SELECT *  
2 FROM R  
3 WHERE (cond1) AND (cond2) AND (cond3) ...
```

- choose one of the conditions, and get all tuples satisfying that using one of the previous techniques
- test the other conditions in memory for each.
- try to choose condition for which can find a plan of least cost
- Cost of execution is cost of that plan.

(if you have a multikey index on more than one of the conditions, can use that too)

“and” query by intersection

```
1 SELECT *  
2 FROM R  
3 WHERE (cond1) AND (cond2) AND (cond3) ...
```

- Use an index on each, get RIDs for each
- Take intersection of RIDs
- Go get data
- Cost = all index scans, plus cost of intersection (in memory?), plus getting data.

“or” query by intersection

```
1 SELECT *  
2 FROM R  
3 WHERE (cond1) OR (cond2) OR (cond3) ...
```

- can do union if all have indexes
- but if any one needs a full scan, no point